

Relatório de Arquitetura da Ravena AI

1. Introdução

Este relatório detalha a arquitetura do projeto Ravena AI, com base na análise dos arquivos e estrutura de pastas localizados na pasta `ravena ai core infrastructure` do Google Drive. O objetivo é fornecer uma compreensão clara dos componentes do sistema, suas funcionalidades e interconexões, bem como documentar o versionamento de arquivos-chave para futuras manutenções e atualizações.

2. Visão Geral da Arquitetura

A Ravena AI é um sistema complexo e modular, projetado para ser uma assistente de IA soberana, capaz de operar em diferentes modos (offline total, híbrido) e integrar diversas capacidades cognitivas. A arquitetura é centrada em um `engine` (motor de orquestração) que coordena as interações entre módulos especializados, como o `brain` (cérebro cognitivo), `RAG` (Retrieval-Augmented Generation), gerenciamento de `memória`, `segurança` e `API` de comunicação.

Os principais pilares da arquitetura incluem:

- **Modularidade:** Componentes bem definidos e desacoplados, facilitando a manutenção e a adição de novas funcionalidades.
- **Soberania Digital:** Capacidade de operar de forma autônoma, com controle sobre o uso de recursos externos e modelos locais.
- **Cognição Aumentada:** Utilização de RAG para integrar conhecimento externo e validação de veracidade.
- **Aprendizado Contínuo:** Mecanismos para aprender com erros e refinar seu conhecimento.
- **Segurança:** Implementação de protocolos de segurança e validação de código.

3. Componentes e Módulos Detalhados

A seguir, são detalhados os principais módulos e arquivos identificados, com suas respectivas funcionalidades e, quando aplicável, as versões identificadas durante o processo de versionamento.

3.1. API e Gerenciamento de Modo

- **api (Modo Soberania Digital V0.1).py** : Este arquivo define a API FastAPI da Ravena AI, servindo como interface principal para interação com o sistema. Ele expõe endpoints para perguntas, aprendizado, feedback, histórico, gerenciamento de categorias e controle do modo de operação. A versão **V0.1** indica uma fase inicial de implementação do modo de soberania digital.
- **gerenciador_modos_soberano (Ravena AI 1.7).py** : Responsável por controlar os modos de operação da Ravena (Offline Total, Híbrido, Externo) e seus respectivos níveis de soberania. Ele gerencia o acesso a ferramentas externas e registra eventos de auditoria. A versão **1.7** sugere um estágio avançado de desenvolvimento para este componente crítico.

3.2. Motor de Orquestração (Engine)

- **engine (Motor de Orquestração V2.0).py** : Este é o motor central que orquestra as operações da Ravena, integrando diferentes módulos e funcionalidades. Ele coordena a interação com o modelo fine-tuned, a ponte de inteligência e outras ferramentas. A versão **V2.0** indica uma revisão significativa na arquitetura de orquestração.
- **engine (Motor de Orquestração Completo).py** : Uma versão mais completa do motor de orquestração, possivelmente com funcionalidades adicionais ou otimizações em relação à V2.0. A presença de múltiplas versões do **engine.py** sugere um desenvolvimento iterativo e aprimoramento contínuo.
- **engine (Motor de Orquestração Completo - Cópia).py** : Uma cópia do motor de orquestração completo, provavelmente para fins de backup ou experimentação.

3.3. Cérebro Cognitivo (Brain) e Clarividência

- **brain (Ponte de Inteligência V1.3).py** : Implementa a Ponte de Inteligência, que cruza regras técnicas (DevDocs) com exemplos práticos (Codewars) para validar o conhecimento e evitar alucinações. A versão **V1.3** indica um estágio inicial de desenvolvimento desta funcionalidade.
- **brain_clarividencia (Cérebro V1.5).py** : Este módulo representa uma evolução do cérebro cognitivo, incorporando a funcionalidade de Clarividência Externa. Ele utiliza um **SearchAgent** para descobrir e aprender novos conhecimentos na web, triangulando fontes e filtrando informações irrelevantes. A versão **V1.5** demonstra a adição de capacidades de busca e síntese de informações externas.
- **clarividencia (Agente de Síntese Dialética V1.0).py** : Atua como um agente de pesquisa autônomo, realizando buscas trianguladas e aplicando um Filtro do Juiz para descartar informações irrelevantes ou de baixa qualidade. A versão **V1.0** indica a primeira implementação deste agente.

3.4. RAG (Retrieval-Augmented Generation)

- **rag_best_practices (RAG para Best Practices V1.7).py** : Este módulo implementa a funcionalidade de Retrieval-Augmented Generation (RAG) para integrar conhecimento externo, como best practices de segurança (OWASP), frontend, backend e DevOps, utilizando o ChromaDB. Ele alimenta o Protocolo Lockdown com padrões modernos e valida o código contra padrões da indústria. A versão **V1.7** indica um desenvolvimento maduro para este componente.
- **rag_best_practices (RAG para Best Practices V1.7 - Cópia).py** : Uma cópia do módulo RAG para best practices, provavelmente para fins de backup ou experimentação.
- **rag_advanced (RAG Avançado V2.0).py** : Uma versão avançada do RAG, que provavelmente inclui funcionalidades aprimoradas para recuperação e geração de informações, como a integração com o módulo **clarividencia** para busca em tempo real e o **modulo_juiz** para validação de veracidade. A versão **V2.0** sugere uma evolução significativa em relação à versão anterior.

3.5. Segurança e Validação

- **juiz_universal (Protocolo Lockdown V2.2).py** : Este módulo implementa o Protocolo Lockdown, responsável por avaliar a conformidade do código e das operações com as políticas de segurança e as melhores práticas. Ele registra violações e pode acionar bloqueios em caso de risco. A versão **V2.2** indica um protocolo de segurança robusto e atualizado.
- **conftest (Configuração de Testes de Segurança V1.0).py** : Arquivo de configuração para testes de segurança, provavelmente utilizado para validar a conformidade do sistema com as políticas definidas no Protocolo Lockdown. A versão **V1.0** indica a primeira versão desta configuração.
- **conftest (Configuração de Testes de Segurança V2.0).py** : Uma versão atualizada da configuração de testes de segurança, possivelmente com novos testes ou melhorias na cobertura.

3.6. Memória e Conhecimento

- **memoria (Memória de Curto Prazo V1.0).py** : Gerencia a memória de curto prazo da Ravena, armazenando o histórico de conversas e interações recentes para manter o contexto. A versão **V1.0** indica a implementação inicial deste módulo.
- **memoria (Memória Curta).py** : Uma versão alternativa ou cópia do módulo de memória de curto prazo.
- **conhecimento_base (Base de Conhecimento JSON).py** : Este arquivo gerencia a base de conhecimento persistente da Ravena, armazenando perguntas e respostas em formato

JSON. Ele permite adicionar, remover, obter e carregar conhecimentos, além de registrar feedback. A ausência de uma versão explícita no nome sugere que esta é a implementação principal ou mais estável da base de conhecimento.

- **intencao_categoria (Classificação de Intenção - Embeddings).py** : Responsável por classificar a intenção das perguntas do usuário e categorizar os dados, utilizando embeddings para calcular a similaridade semântica. A versão **V1.0** indica a implementação inicial desta funcionalidade.

3.7. Outros Componentes

- **cripto (Utilitário Base64).py** : Um utilitário simples para criptografia e descriptografia usando Base64.
- **omega (Orquestrador Central V1.0).py** : Um orquestrador central, provavelmente uma versão anterior ou alternativa do **engine.py** , responsável por coordenar as operações. A versão **V1.0** indica a primeira implementação deste orquestrador.
- **ravena_model (Modelo Fine-Tuned LoRA).py** : Contém o modelo Ravena Fine-Tuned com suporte a LoRA, utilizado para gerar respostas. Este é um componente chave para a capacidade de geração de texto da Ravena.
- **cognitive_ingestion (Ingestão Cognitiva V1.0).py** : Módulo responsável pela ingestão de dados e informações para a base de conhecimento da Ravena. A versão **V1.0** indica a primeira implementação deste processo.
- **cognitive_ingestion (Ingestão Cognitiva V2.0).py** : Uma versão atualizada do módulo de ingestão cognitiva, provavelmente com melhorias no processo de coleta e processamento de dados.
- **ravena_modular_main (Main Modular V1.0).py** : O ponto de entrada principal para a arquitetura modular da Ravena, coordenando a inicialização e interação dos diferentes módulos. A versão **V1.0** indica a primeira implementação desta estrutura modular.

4. Versionamento de Arquivos na Pasta **99_Backup_Versoes**

Durante a análise, os seguintes arquivos foram identificados e renomeados na pasta **99_Backup_Versoes** para refletir suas versões e funcionalidades:

Nome Original do Arquivo	ID do Arquivo no Google Drive	Nome Versionado no Google Drive
chroma_persistence.py	1lcUCBIP3X19wSKPlg41EgwtHEU	chroma_persistence (Persistência ChromaDB V1.0).py

rag_advanced.py	12yiaCNikVbX67pGl6M_78f4RHU	rag_advanced (RAG Avançado V2.0).py
conftest.py	1F23uxox090909090909090909090909	conftest (Configuração de Testes de Segurança V1.0).py
omega.py	1sCaSQ3E090909090909090909090909	omega (Orquestrador Central V1.0).py
conftest.py	10AUK8Qe090909090909090909090909	conftest (Configuração de Testes de Segurança V2.0).py
cognitive_ingestion-2.py	1kboU47p090909090909090909090909	cognitive_ingestion (Ingestão Cognitiva V2.0).py
cognitive_ingestion-1.py	18tW8dig090909090909090909090909	cognitive_ingestion (Ingestão Cognitiva V1.0).py
ravena_modular_main.py	1QN8it1i09090909090909090909090909	ravena_modular_main (Main Modular V1.0).py
juiz_universal.py	1awMWkM50909090909090909090909090909	juiz_universal (Protocolo Lockdown V2.0).py
memoria.py	18nol9hT0909090909090909090909090909	memoria (Memória de Curto Prazo V1.0).py
conhecimento_base.py	1VQe8aAX0909090909090909090909090909	conhecimento_base (Base de Conhecimento JSON).py
intencao_categoria.py	187I73-t09090909090909090909090909090909	intencao_categoria (Classificação de Intenção - Embeddings).py
memoria.py	1wu4diVu09090909090909090909090909090909	memoria (Memória Curta).py
juiz_universal.py	1QUZpuES09090909090909090909090909090909	juiz_universal (Protocolo Lockdown V2.2).py
conhecimento_base.py	1D9B3x9Jg1AVA8ijqi6L9gluljafzbja3	conhecimento_base (Base de Conhecimento JSON).py
intencao_categoria.py	1tj87BZ532XfQ9bJn9S8xLZ8iXQPB1YMY	intencao_categoria (Classificação de Intenção - Embeddings).py
ravena_model.py	1Nh01FVybvko8sdTfyBlvL8w3k3DnsG5O	ravena_model (Modelo Fine-Tuned LoRA).py

engine.py	1Lezn1kFRszurHdPw9ztr5WtPjTvUz9tq	engine (Motor de Orquestração V2.0).py
engine.py	1Q8iRSE-eUBmAEOwH7p8agDGA2DKdVqmS	engine (Motor de Orquestração Completo).py
engine.py	1UDKCSSXbg3Px7wv5I2QxR-XcVmXoQS8y	engine (Motor de Orquestração Completo - Cópia).py
cripto.py	1SQLk-KG5c44Smxcx9woMk-GZ_9dj7zUb	cripto (Utilitário Base64).py
api.py	1_7d-4TQWURsja6aPojWeWxRNaO5Hxix1	api (Modo Soberania Digital V0.1).py
gerenciador_modosoberano.py	1OEKOvEynHN-J1UlrJH6bDLfUGncPLm9Z	gerenciador_modosoberano (Ravena AI 1.7).py
rag_best_practices.py	14j6kPv_auRRsygsyMFUhrLcxPRyCkCA	rag_best_practices (RAG para Best Practices V1.7).py
rag_best_practices.py	1fRbve-805WzboGhvnOw9S18zV_kERjcp	rag_best_practices (RAG para Best Practices V1.7 - Cópia).py
brain_clarividencia.py	1vV1s8kcl1G4B-aamr2onJ3DXIUT_9A1r	brain_clarividencia (Cérebro V1.5).py
clarividencia.py	1zkM-CK5b5CNQ3r0wNh9YObxmyXT0aIQA	clarividencia (Agente de Síntese Dialética V1.0).py
brain.py	1VZb26uutcgMpAP4uLGiiZFJ_WwB CXE7R	brain (Ponte de Inteligência V1.3).py
api.py	1iDao_iiz9rWEb1qixrl0g2RONLZdMrS8	api (API de Controle e Ferramentas).py
oraculo_refactor.py	1J0E7kfjC8_T25HSn91NbElxHiGo21IMG	oraculo_refactor (Refatorador Soberano).py

5. Conclusão

A arquitetura da Ravena AI é robusta e bem organizada, com uma clara separação de responsabilidades entre seus módulos. O uso de versionamento explícito nos nomes dos arquivos, especialmente na pasta de backup, é uma prática excelente que facilita o rastreamento de mudanças e a recuperação de versões anteriores. A integração de componentes como RAG, Clarividência e o Protocolo Lockdown demonstra um sistema sofisticado, focado em autonomia, segurança e aprendizado contínuo. As informações coletadas neste relatório servirão como base para quaisquer futuras alterações ou atualizações no projeto.

6. Referências

- [1] Google Drive - Pasta ravena ai core infrastructure
- [2] DocumentoMestreArquitetura_V3.2.1.md
- [3] RELATORIOTECNICOIARAGVISAO.md
- [4] RELATORIOTECNICOINFRAESTRUTURA_HARDWARE.md
- [5] relatorioavaliacaoarquitetura_360.md
- [6] relatorioautonomiaravana.md